

SPEC 2111

Deep Learning and Neural Networks

Course Roadmap

PART-1: Introduction, Foundations & Essentials

- Week 1 – Introduction & Course Overview
- Week 2 – Linear Regression
 - Regression concepts, gradient descent, loss functions
- Week 3 – Logistic Regression
 - Classification, sigmoid, cross-entropy loss

PART-2: Neural Networks, Deep Learning and Training

- Week 4 – Neural Network Essentials
 - Neurons, layers, forward propagation, activation functions
- Week 5 – Training, Refining Backpropagation and Deep Neural Networks (DNNs), Backpropagation, chain rule, gradient descent optimization
- Week 6 – Preventing Overfitting, Activation Functions, Optimizers

PART-3: Specialized Architectures

- Week 7 – Convolutional Neural Networks (CNNs)
 - Convolutions, pooling, image applications
 - Assessment-1
- Week 8 – Representations & Transfer Learning (and Test)
 - Feature learning, pre-trained models, fine-tuning
- Week 9 – Recurrent Neural Networks & LSTM
 - RNNs, LSTM/GRU, sequence modeling
- Week 10 – Attention & Transformers
 - Attention mechanism, transformer architecture, NLP applications

PART-4: Reinforcement Learning & Projects

- Week 11 – Reinforcement Learning
 - Assessment-2
- Week 12 - Project Presentations & Discussions (Project Due)

Week-5 / Agenda

Theme: Improving Performance

- Theory
 - Recap - Week 6
 - CNNs
 - Motivation: High-Dimensional Data
 - CNN Architecture Overview
 - Core CNN Components
 - Deep CNNs and Residual Connections
 - Efficient Architectures: Inception
 - Pre-training and Transfer Learning
 - Lab Work - notebook to be checked on your own time
- Assessment
 - Starts at ~10:10am, 45min, 1 attempt to submit
 - Open only lecture notes & lecture notebooks



Loss and Activation Functions, Reducing Overfitting, Optimizers

- Activation functions
 - Tanh, ReLU
 - sigmoid: $[0,1]$, hidden or output;
 - tanh: $[-1,1]$, mostly hidden;
 - ReLU: $\max(0,x)$, mostly hidden, sparse;
 - softmax: outputs probabilities summing to 1, used in multi-class output.
- Solving overfitting
 - Regularization (L1/L2)
 - Early Stopping
 - Dropout
 - Good initialization
 - Proper data management
- Other optimization methods
 - SGD
 - One or more training examples than entire dataset
 - Adagrad
 - Adaptive learning rate per parameter
 - Adam
 - Momentum + Adaptive Learning Rate

Motivation: Why Special Architectures for High-Dimensional Data

- Fully Connected Networks
 - In a fully connected (dense) network:
 - Every hidden neuron receives all input features
 - Each neuron can learn global patterns across the entire input
 - This makes the model very flexible

The Problem with High-Dimensional Data (like images)

- For images, fully connected networks have important drawbacks:
 - Very expensive computationally
 - Every neuron connects to all pixels
 - Too many parameters
 - Large images create millions of weights
 - No notion of spatial locality
 - The network treats pixels far apart the same as neighboring pixels
 - Misses reusable local patterns
 - Important features like edges or corners appear in many places in an image

Problem 1: Computational Expensiveness

How many weights (or parameters) are required for a 256×256 three colour image that is to be fully connected to a hidden layer with 128 neurons?

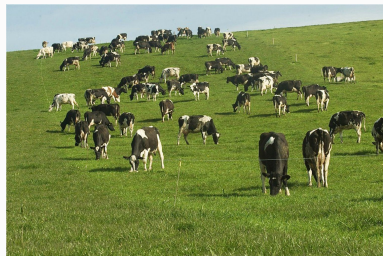
If it's a 256×256 image with 3 color values per pixel (R, G, B), hence 3-channels

- Input is a 3D tensor of shape (256,256,3)
 - input size: $256 \times 256 \times 3 = 196,608$ neurons.
- Fully connected to 128 neurons
 - $196,608 \times 128 = 25,165,824$ weights
 - Bias terms: each hidden neuron also has a bias: 128 biases
- Final Total Parameters: $25,165,824 + 128 = 25,165,952$ weights in one layer!



This is computationally demanding, very data-intensive and increases the risk of overfitting

Problem 2 - Spatial Locality and Local Patterns



- Fully connected network → **look at the whole photo at once** → sensitive to shifts, rotations, or multiple objects
- CNN → **look at pieces first** → combine them → robust to movement and multiple objects

- **Problem with fully connected networks for images:**
 - Every neuron treats **all pixels equally**, whether they are neighbors or far apart.
 - This makes it **hard to recognize patterns** like edges, corners, or textures, because the network has to look at the **whole image at once**.
- **Why local patterns matter:**
 - Images have **local structure**
 - By focusing on **small neighborhoods of pixels**, network can **learn important local features** first
- **Building global understanding from local parts:**
 - Once local patterns detected, network can **combine them to understand larger patterns**
 - more **efficient and natural** than trying to analyze whole image at once.

Convolutional layers were developed to address these shortcomings by using

- Locality
- Translation Invariance

Locality & Translation Invariance in CNNs

1. Locality (Focus on small neighborhood first)

- Images have local patterns (edges, corners, textures) that are meaningful.
- CNNs detect features in small patches of pixels rather than the whole image.
- Benefits:
 - Reuses the same detectors across the image
 - Reduces the number of parameters
 - Preserves spatial structure

2. Translation Invariance

- If something appears in one location, it could appear anywhere else in the image.
- A horizontal edge in the:
 - Top-left corner, center, bottom-right is still a horizontal edge.
- Achieved via:
 - Convolution (feature detectors applied across the image)
 - Pooling (captures feature presence, not exact position)

By focusing on local patterns and being translation-invariant, CNNs are both **computationally efficient** and **more accurate**.

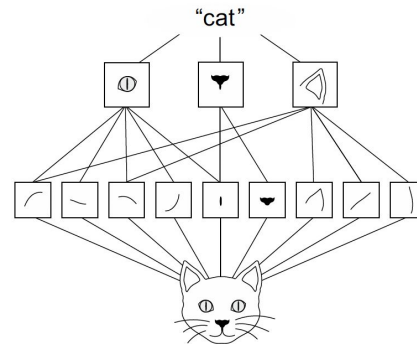
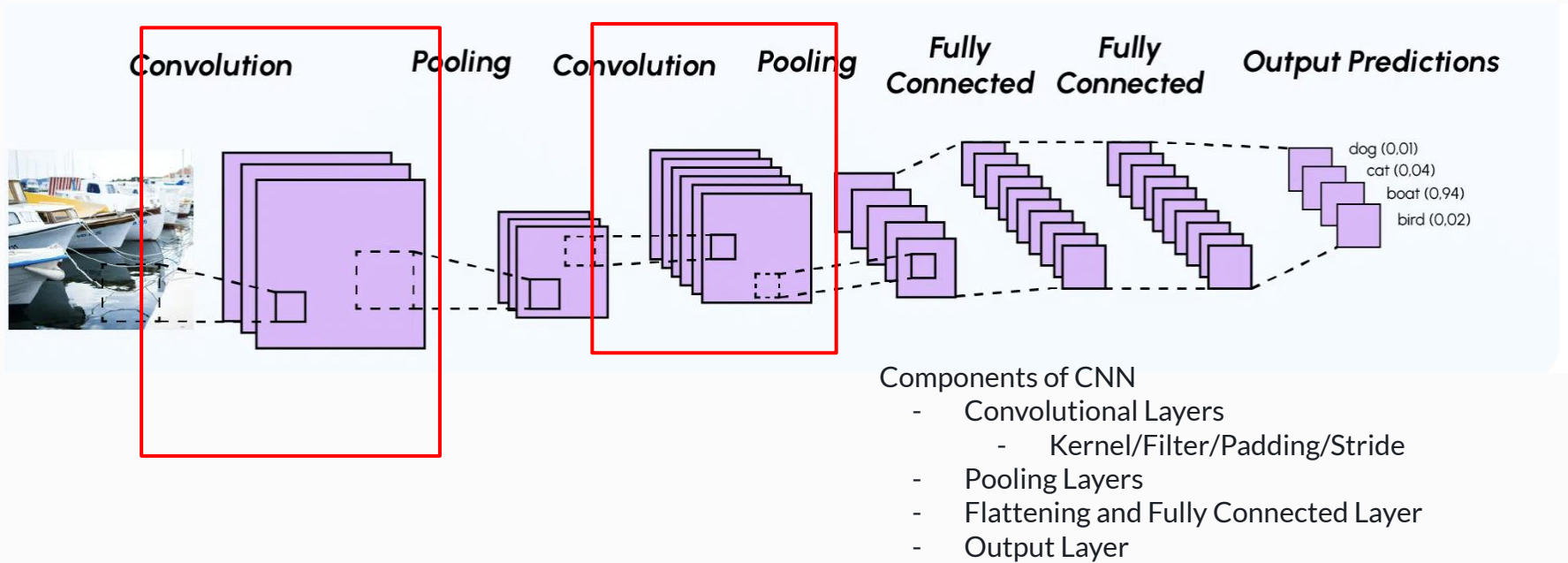


Figure 8.2 The visual world forms a spatial hierarchy of visual modules: elementary lines or textures combine into simple objects such as eyes or ears, which combine into high-level concepts such as "cat."

Bottom-Up Feature Learning

CNN Architecture - Convolutional Layers



Convolutional Layers and Filters

- A convolutional filter is a small square matrix of numbers
- A convolutional layer is composed of one or more convolutional filters
- By choosing the numbers in a filter and applying the filter to an image, different features of the image can be detected

This filter can detect horizontal lines!

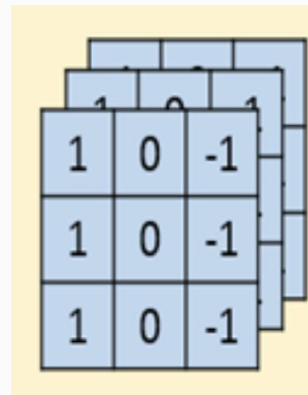
	1		1		1
	0		0		0
	-1		-1		-1

This filter can detect vertical lines

	1		0		-1
	1		0		-1
	1		0		-1

A **horizontal edge detector** is designed to detect edges that run **left-right** (i.e., horizontal lines).

To do that, it looks for **vertical changes** in pixel intensity.



Applying a convolutional filter to an image is called the convolution operation

Input

0	0	0	0	0	0	0
5	5	5	5	5	5	5
10	10	10	10	10	10	10
15	15	15	15	15	15	15
10	10	10	10	10	10	10
5	5	5	5	5	5	5
0	0	0	0	0	0	0

Filter

1	1	1
0	0	0
-1	-1	-1

Output

0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0
0	0	0	0	0

Applying a convolutional filter to an image is called the convolution operation

Input

0	0	0	0	0	0	0	0
5	5	5	5	5	5	5	5
10	10	10	10	10	10	10	10
15	15	15	15	15	15	15	15
10	10	10	10	10	10	10	10
5	5	5	5	5	5	5	5
0	0	0	0	0	0	0	0

Filter

1	1	1
0	0	0
-1	-1	-1

Output

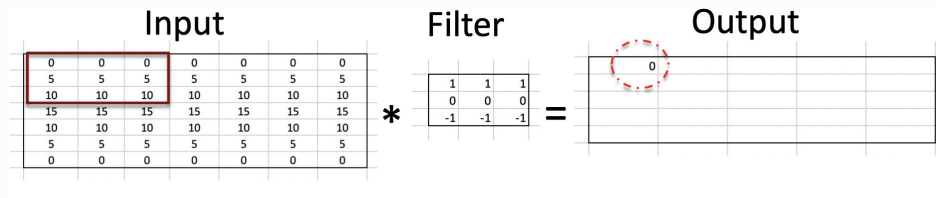
*

=

The convolution operation

- “Overlay” the filter onto the top-left of the image
- Multiply matching elements and add up: $0*1 + 0*1 + 0*1 + 5*0 + 5*0 + 5*0 + 10*-1 + 10*-1 + 10*-1 = -30$
- Run through a ReLU:
 - $\max(0, -30) = 0$.
 - This number is the top-left cell of the output

Applying a convolutional filter to an image is called the convolution operation



Slide the window one step to the right and repeat this process to get the second number of the output

The convolution operation

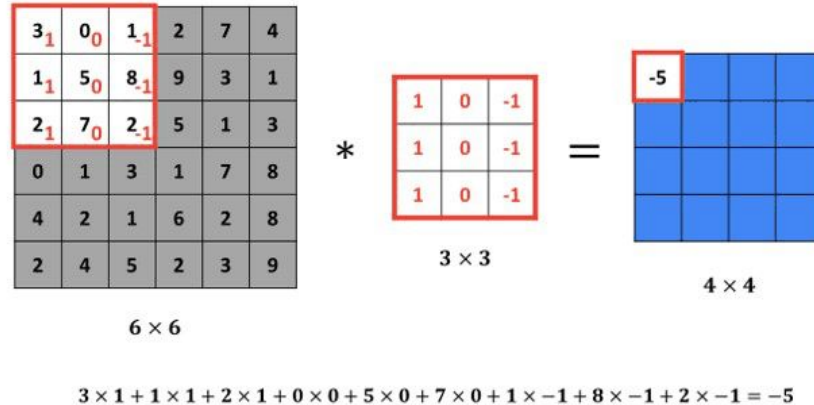
- “Overlay” the filter onto the top-left of the image
- Multiply matching elements and add up: $0*1 + 0*1 + 0*1 + 5*0 + 5*0 + 5*0 + 10*-1 + 10*-1 + 10*-1 = -30$
- Run through a ReLU:
 - $\max(0, -30) = 0$.
 - This number is the top-left cell of the output

Applying a convolutional filter to an image is called the convolution operation

Input								Filter			Output					
0	0	0	0	0	0	0	0	1	1	1	0	0	0	0	0	0
5	5	5	5	5	5	5	5	0	0	0						
10	10	10	10	10	10	10	10	-1	-1	-1						
15	15	15	15	15	15	15	15									
10	10	10	10	10	10	10	10									
5	5	5	5	5	5	5	5									
0	0	0	0	0	0	0	0									

When first row completed, move to the start of the second row and continue as before till you reach the bottom-right corner

Convolution - Sliding a filter across the image



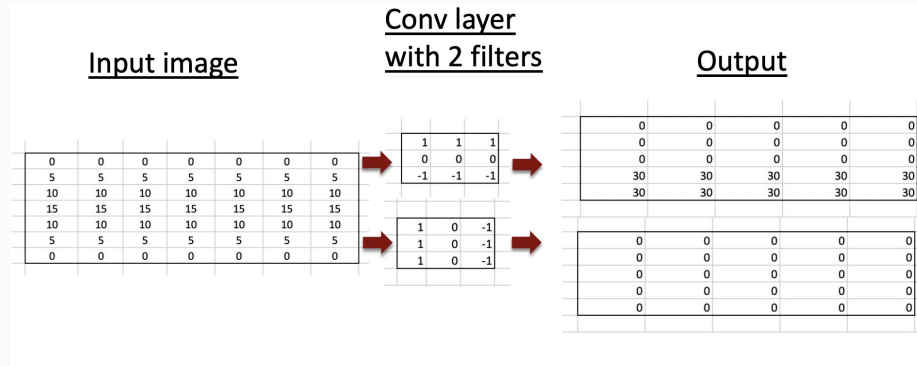
Imagine a **magnifying glass** you slide over the image.

- You don't look at the whole image at once.
- You only analyze **small patches** (e.g., 3x3 or 5x5 pixels).
- The **same detector** is applied everywhere.

- convolution operation is dot product between filter and patch.
- After each convolution, apply an activation function to introduce non-linearity and allow network to learn complex features.
- ReLU is commonly used due to its effectiveness in accelerating convergence during training.

A convolutional layer is composed of one or more convolutional filters

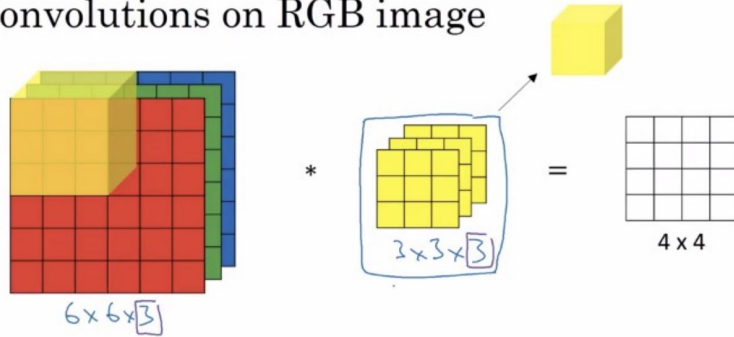
- Each filter can be thought of as a specialist for detecting a particular feature (e.g., a horizontal line, an arc, a vertical line)



These two 5x5 matrices can be represented as a tensor of shape $5 \times 5 \times 2$ or $2 \times 5 \times 5$

Applying a Convolutional Filter to a color image

Convolutions on RGB image



TERMINOLOGY

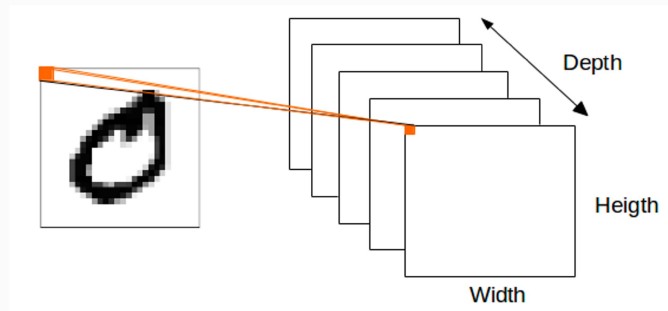
- A **channel** is one layer of information in the data (used for both input, filters etc).
 - Example: a color image
 - Height $\times$ Width $\times$ Channels
 - 1 channel for red, 1 for blue, 1 for green
 - So the image is three stacked 2D matrices.
- A **filter** is a stack of kernels, one for each input channel, that together detect a feature across all channels and produce one output feature map.
 - Filter depth = input depth
- A **kernel** is a small matrix of weights that is applied to one input channel
- A **patch** is the small region of the input the filter is currently looking at.
- **Feature map** is the output of a filter after convolution across the entire input

How are we supposed to come up with the numbers in each filter?

- Filters used to be hand-designed
 - Researchers manually created filters to detect edges, corners, textures, etc.
 - Designing effective filters was hard and time-consuming
- Filters as weights
 - Convolutional filters are just weights, like the weights in a fully connected neuron
 - Instead of designing them, we can learn them from data
- Filters = neurons
 - Each filter behaves like a special neuron:
 - It “activates” when its preferred pattern appears in the input
 - Can be trained using backpropagation and gradient descent
- Entire CNN learns automatically
 - same machinery used for neural networks applies: neurons, layers, loss functions, gradient descent
 - network learns the best filters to detect meaningful features for the task

Convolutional Layer Parameters

- Convolutional layer parameters include
 - kernel **width** (W) and **height** (H)
 - spatial dimensions of the learnable kernels
 - **depth**
 - number of filters
- Note that number of input channels is NOT a layer parameter you choose.
 - It is **determined by the input to the layer** (the previous layer's output).
 - **a filter always spans the full input depth.**
 - So if the input has 3 channels and kernel size is 3×3 :
 - Filter size = $3 \times 3 \times 3$



Practical Tips

Number of Layers

- Shallow CNNs (3–5 conv layers)
 - for small/simple datasets like MNIST or CIFAR-10.
- Moderate CNNs (5–10 conv layers)
 - common for medium datasets.
- Deep CNNs (50+ layers)
 - usually use residual blocks (ResNet) or Inception modules to avoid vanishing gradients

Rule of thumb: start small, then increase depth only if dataset and task require it.

Kernel (Filter) Size

- Common kernel sizes: 3×3 , 5×5 , 7×7
- Small kernels (3×3) are popular today:
 - Less parameters $\rightarrow$ easier to train
- Large kernels (5×5 , 7×7) $\rightarrow$ capture bigger patterns in one layer, but more parameters.

Rule of thumb: start with 3×3 for most layers, use larger only at early layers if needed.

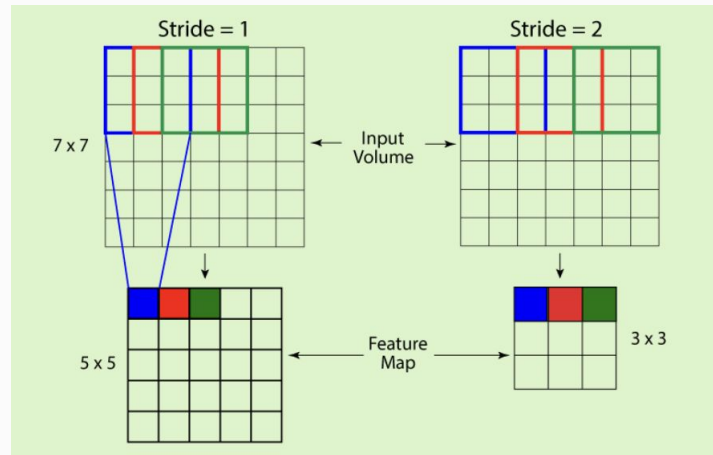
Number of Filters (Depth)

- Start small in early layers (e.g., 32–64)
 - capture simple patterns
- Increase as you go deeper (e.g., 128, 256, 512)

Convolutional Layer Parameters

Stride

- how far the filter moves for each application
- filter (kernel) as a window sliding over the image.
 - Stride = 1 → move 1 pixel at a time
 - Stride = 2 → move 2 pixels at a time (faster, less overlap)
- Stride controls the overlap between patches.
- Stride 1 means maximum overlap.
 - So stride 1 is typically used when detail matters.
- Larger stride reduces the amount of computation and the size of the output.
 - Some spatial information might be lost
 - Small features may disappear



Convolutional Layer Parameters

Padding

- Padding adds extra pixels around the edges of the input image (usually zeros).
- These extra pixels allow filter to fully cover the edges of the image.
 - Without padding, pixels at the edges are used less in convolution → edge features may be lost
- With padding:
 - You can keep output feature map same size as input
 - Feature maps don't shrink quickly

0	0	0	0	0	0	0
0	60	113	56	139	85	0
0	73	121	54	84	128	0
0	131	99	70	129	127	0
0	80	57	115	69	134	0
0	104	126	123	95	130	0
0	0	0	0	0	0	0

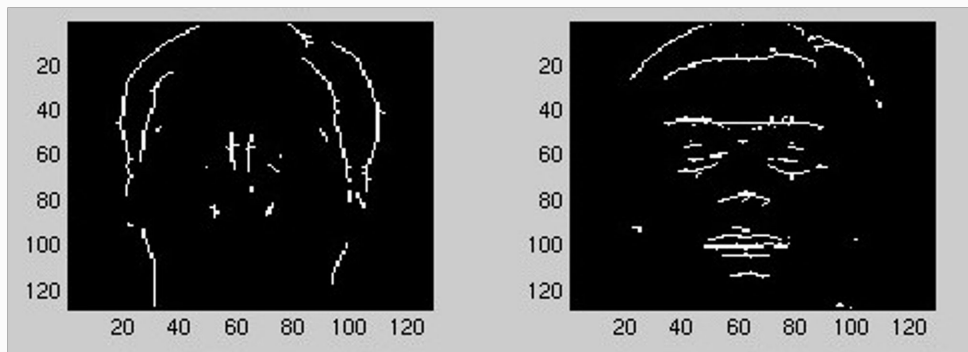
Kernel		
0	-1	0
-1	5	-1
0	-1	0

114				

Type	Description	Example
Valid	No padding; output smaller than input	5×5 → 3×3 with 3×3 filter
Same	Pads input so output size = input size	5×5 → 5×5 with 3×3 filter
Full	Pads so every pixel is fully covered multiple times (less common)	5×5 → 7×7

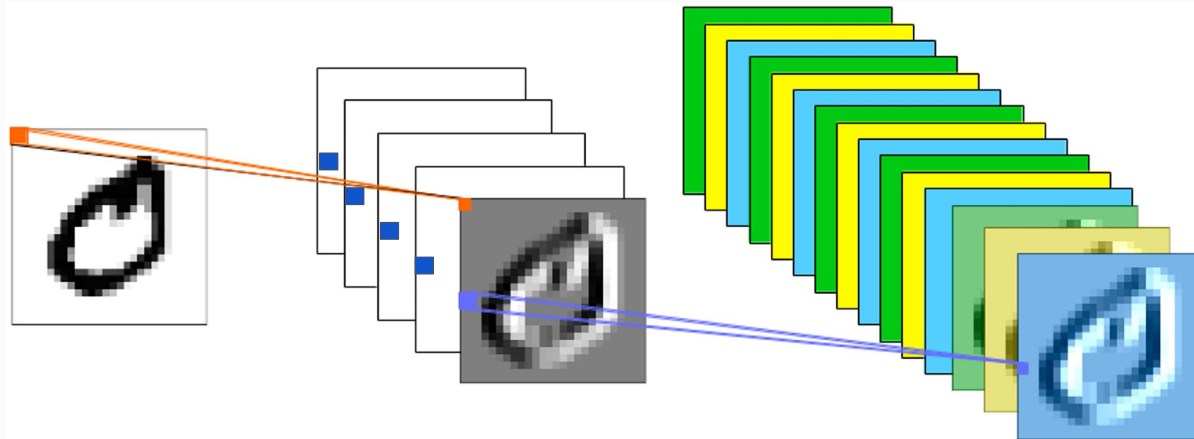
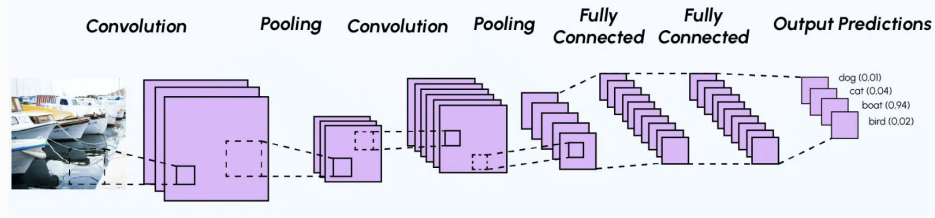
Applying a Feature Detector to a Whole Image

1. Convolution = sliding a filter over the whole image
2. Each position $\rightarrow$ checks only a local patch
3. Same filter is reused everywhere $\rightarrow$ parameter-efficient and translation-invariant
4. Output = feature map showing where the feature appears



Multiple Convolutional Layers

- CNNs usually have multiple convolutional layers arranged in a feedforward manner
- Feature maps from layer k become input channels for layer $k+1$
- Each filter in the next layer sees all previous feature maps
- This allows the network to combine simpler features into more complex patterns



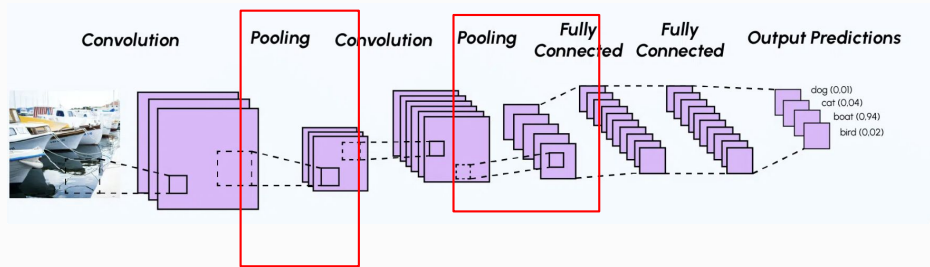
Later conv layers “see” more of the original input than the earlier layers



lines => edges, circles => faces!

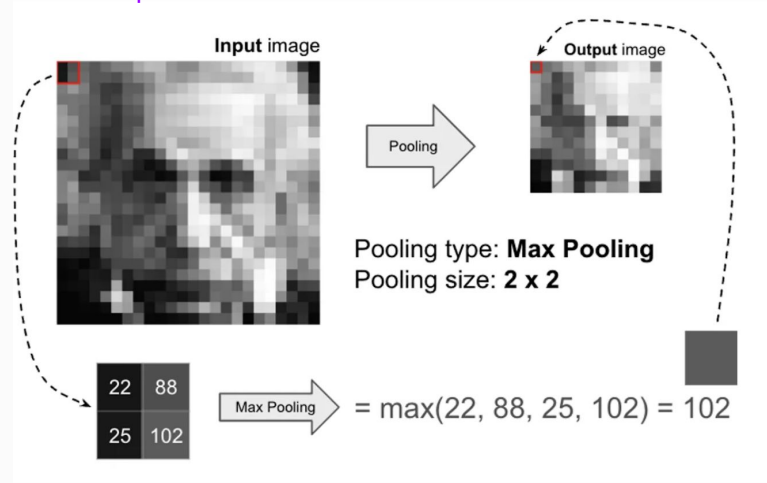
- At first, the network detects small features like edges in tiny patches.
- As layers stack up, neurons “see” larger areas
- Eventually, the network can detect entire objects or large patterns, not just small edges.

CNN Architecture - Pooling Layers

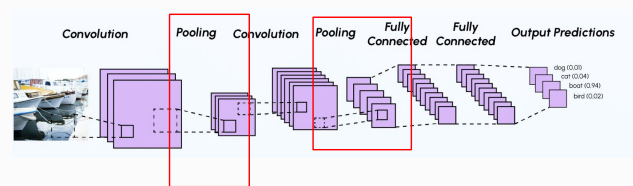


- Pooling effectively summarizes the most prominent features in a particular region of the feature map while discarding less significant details.
- Pooling is used to
 - Reduce feature map size
 - Lower computation and memory use

Keep the most important features, Make the network robust to small translations

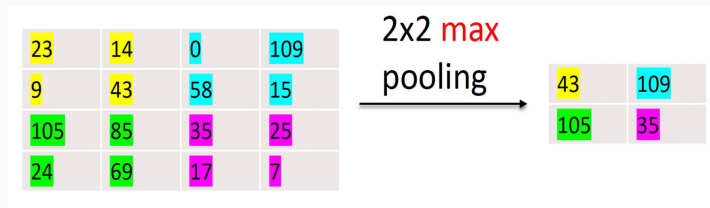


Pooling Subsampling Feature Maps

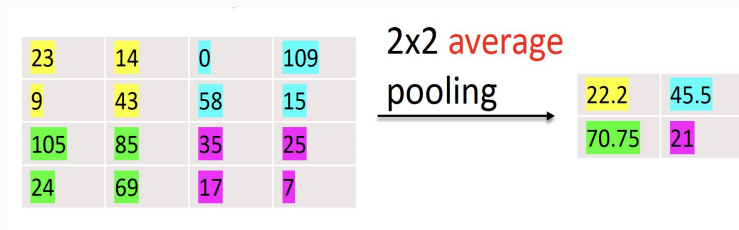


- Two common types

- **Max Pooling:** pick maximum value in a region
 - if any neuron fires strongly → output is high
 - acts like a feature detector
- **Average Pooling:** compute average value
 - output is high only if many neurons fire



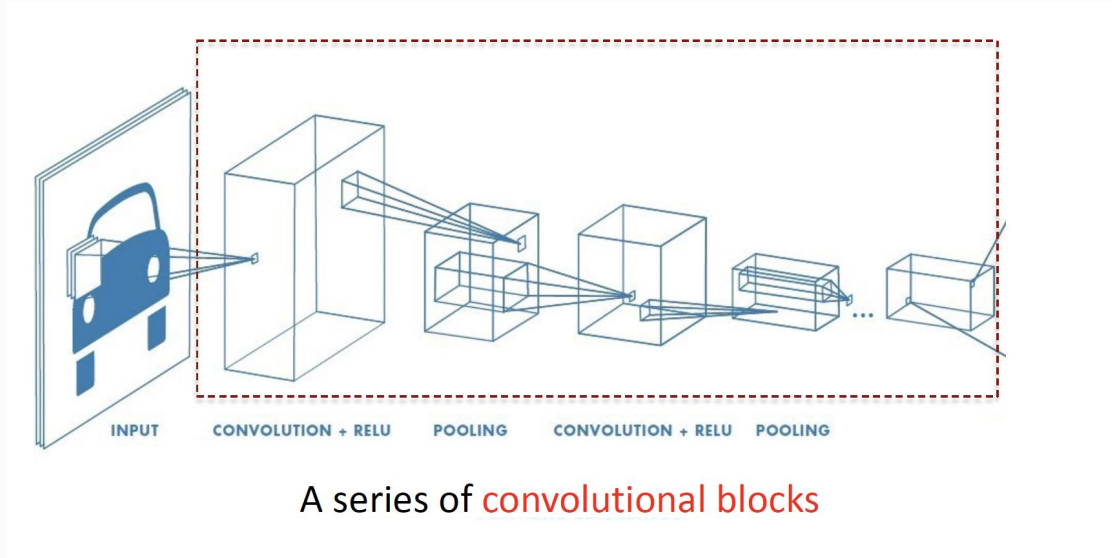
- In *max-pooling*, we take the maximum value from each 2x2 box



- In *avg-pooling*, we take the average value from each 2x2 box

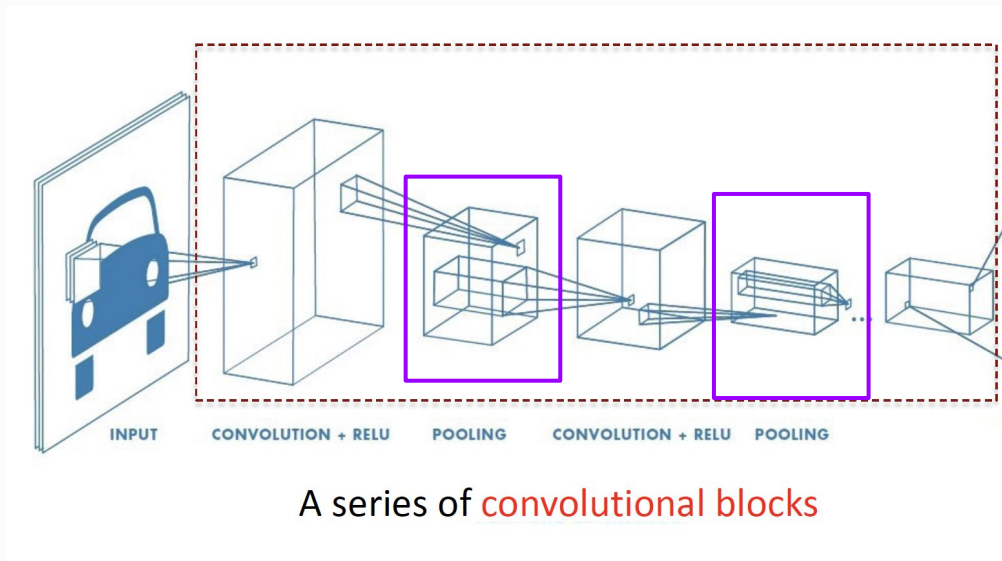
- Reduces the number of entries significantly (e.g., 75% for 2x2 pooling)
- The output from the pooling layer is fed to the next layer

The architecture of a CNN



Each **convolutional block** typically has 1-2 convolutional layers followed by a pooling layer

The architecture of a CNN

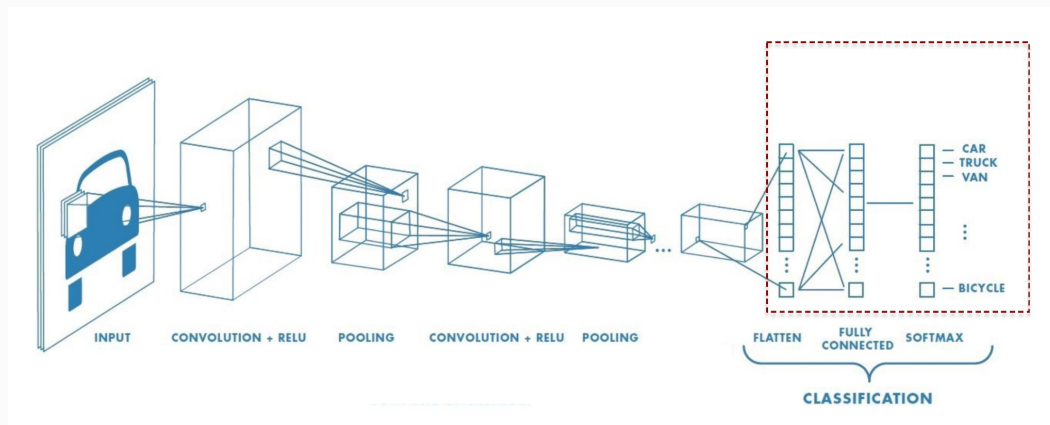


pooling results in smaller spatial size → focus shifts from fine details to global features.

Each block will typically have more depth than the previous block but lower height/width.

- Early layers: "look at lots of small details"
- Deeper layers: "look at many combinations of patterns across the image"
- Increasing depth allows capturing more types of features
- while reducing spatial size focuses computation on higher-level concepts rather than individual pixels.

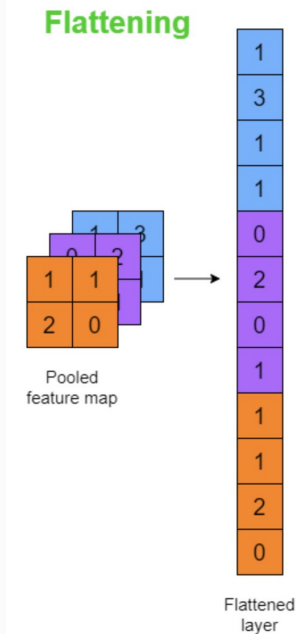
At the end, we flatten the tensor, run it through fully-connected layers, and then the output layer



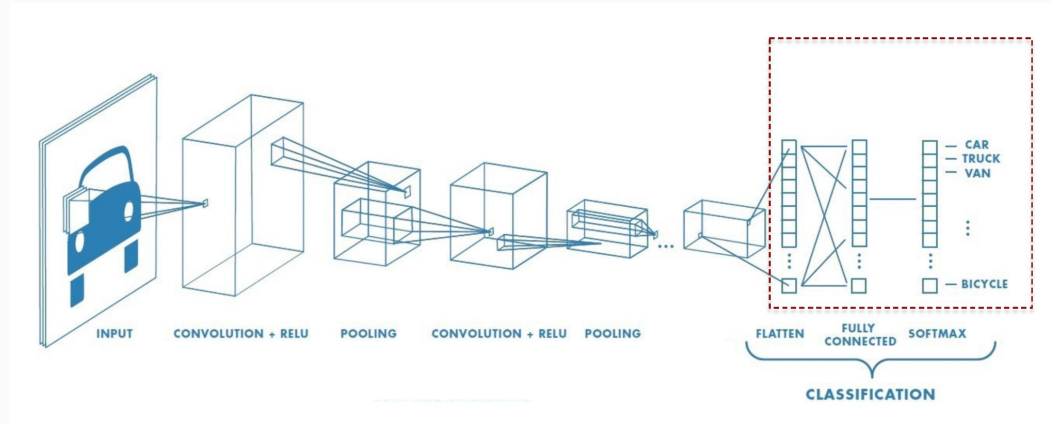
The final tensor gets flattened into a long vector and sent through 0 or more hidden layers to the output layer

Flattening

- Fully connected layer only accepts one-dimensional data.
- we need to flatten the final pooled feature map into a single column that holds the input data for the fully connected layer
- Example: A convolutional layer might output a feature map of shape (batch_size, 14, 14, 32) → batch size: number of examples
 - 14×14 = spatial dimensions
 - 32 = number of feature maps (channels)
- Flatten turns it into (batch_size, $14 \times 14 \times 32$) = (batch_size, 6272)



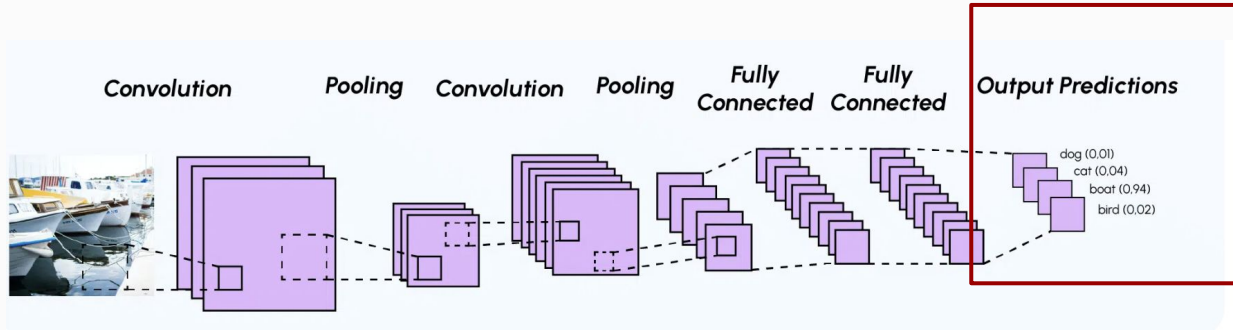
Fully Connected Layers



- Convolutional layers detect local patterns (edges, textures, shapes)
- Fully connected layers combine features for global decision-making

- **fully connected layers** integrate all local features detected by earlier layers to make final decisions.
- Each neuron in these layers is connected to every neuron in the preceding layer.
- can result in explosion of weights if input feature maps are large.
 - Pooling helps control the size of the network and prevents the explosion of weights in fully connected layers.

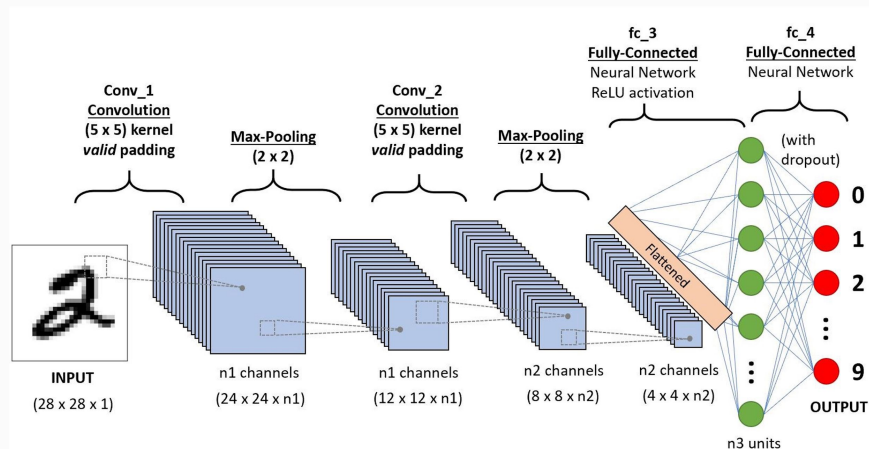
Output Layer



Hidden layers are usually ReLU, but other activations can be used depending on network design or research needs.

- output layer generates predictions based on learned features.
- For classification tasks, this layer typically employs a softmax activation function to produce probabilities for each class.
 - Softmax is not the only option at output.
 - Use output activation depending on whether you want probabilities, independent labels, or continuous values (ex: Age Prediction from a Face Image).
 - Sigmoid is common for binary or multi-label tasks; linear is used for regression.

CNNs for Image Processing

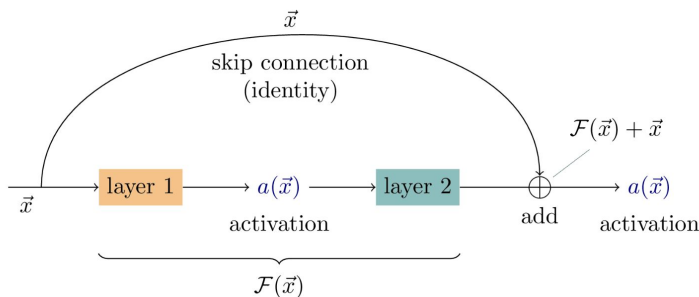


- **Input Layer**
 - Original image (28x28 grayscale MNIST digit).
- **Convolutional Layer**
 - applies multiple filters
 - extracts features like edges, corners, textures.
- **Activation Function**
 - Non-linearity (ReLU) introduces complex pattern detection.
- **Pooling Layer**
 - Reduces spatial dimensions
 - retains key features
 - improves efficiency.
- **Flatten Layer**
 - Converts 2D feature maps into 1D vector for fully connected layers.
- **Fully Connected Layer**
 - Combines extracted features to predict class probabilities.
- **Output Layer**
 - Softmax produces class predictions (e.g., digits 0–9).

Deep CNN Challenges

Why Very Deep Networks Struggle

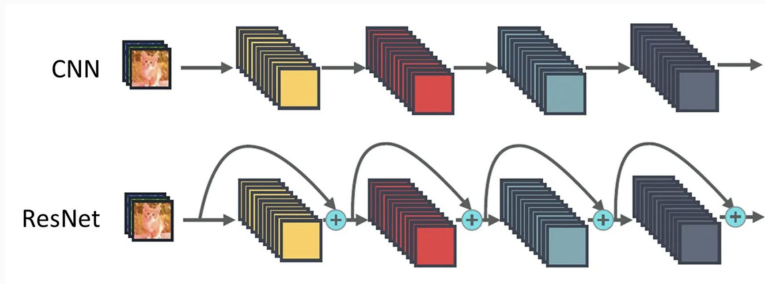
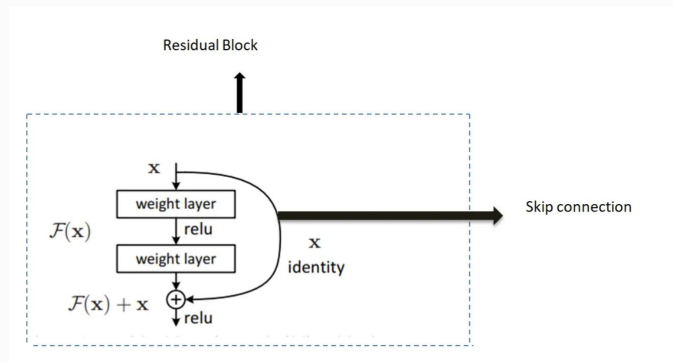
- Stacking more layers $\rightarrow$ can capture more complex features.
- **Problem:** Vanishing gradients
 - Gradients shrink as they backpropagate through many layers.
 - Early layers stop learning effectively.
- Deep CNNs were historically limited by this problem.
- **Skip Connections to the Rescue**
 - Input bypasses one or more layers
 - added directly to later layer outputs.
 - Enables **deeper networks** without vanishing gradient issues.



Think of a skip connection like a **shortcut road**:

- **main road** (convolution layers) learns complex transformations.
- shortcut road (**skip connection**) allows information to travel directly from input to output.
- If transformation layers are not useful, network can simply **pass the input through unchanged**.

Residual Layers



- Residual block = **standard layer(s) + skip connection**
- ResNet = stack of **residual blocks**
- Process:
 - Input $x \rightarrow$ Layer(s) $\rightarrow$ Activation $\rightarrow$ Output
 - Skip connection: x added **before activation** at end of block
- Intuition: Network learns **residuals** (differences) instead of full transformations.
- Gradients can flow backward more easily $\rightarrow$ allows very deep CNNs.

ResNet

- ResNet (**Residual Network**) is a **particular type of CNN**
- It is **built entirely from residual blocks** (layers + skip connections).
- Unlike standard CNNs:
 - ResNet can be **very deep** (50, 101, 152+ layers)
 - Skip connections allow gradients to **flow easily** → solves vanishing gradient problem
- While it's a CNN, it's a **special, highly successful CNN architecture** using residual learning.

Without skip connections, CNNs were typically **10–20 layers deep** at most.
With residuals, networks can now go **much deeper**, 50, 101, 152+ layers.

Residual Layers

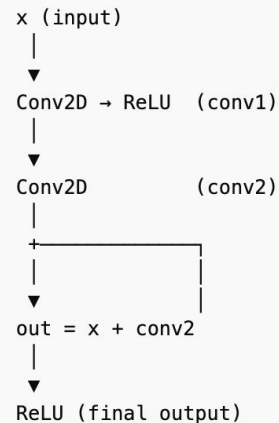
```
import tensorflow as tf
from tensorflow.keras import layers

def residual_block(x, filters, kernel_size=3):
    """
    Simple residual block
    x: input tensor
    filters: number of filters (depth)
    kernel_size: convolution kernel size
    """
    # First convolution
    conv1 = layers.Conv2D(filters, kernel_size, padding='same', activation='relu')(x)

    # Second convolution
    conv2 = layers.Conv2D(filters, kernel_size, padding='same')(conv1)

    # Add skip connection
    out = layers.Add()([x, conv2])    #  $x + F(x)$ 
    out = layers.Activation('relu')(out) # activation after addition

    return out
```



```
inputs = tf.keras.Input(shape=(32,32,64)) # example input

# Apply residual block
x = residual_block(inputs, filters=64)

# Flatten + Dense for classification (optional)
x = layers.Flatten()(x)
outputs = layers.Dense(10, activation='softmax')(x)

model = tf.keras.Model(inputs=inputs, outputs=outputs)
model.summary()
```

Skip Connection

1. Without Skip Connection

Suppose a layer is trying to learn:

$$\text{Output} = H(x)$$

During backpropagation, the gradient is:

$$\frac{dLoss}{dx} = \frac{dLoss}{dH} \cdot \frac{dH}{dx}$$

- In very deep networks, $\frac{dH}{dx}$ is a **product of many small numbers** (weights and derivatives of activations).
- Result: **gradient shrinks toward zero** → early layers learn very slowly → **vanishing gradient problem**.

2. With Skip Connection

Now the layer learns:

$$\text{Output} = F(x) + x$$

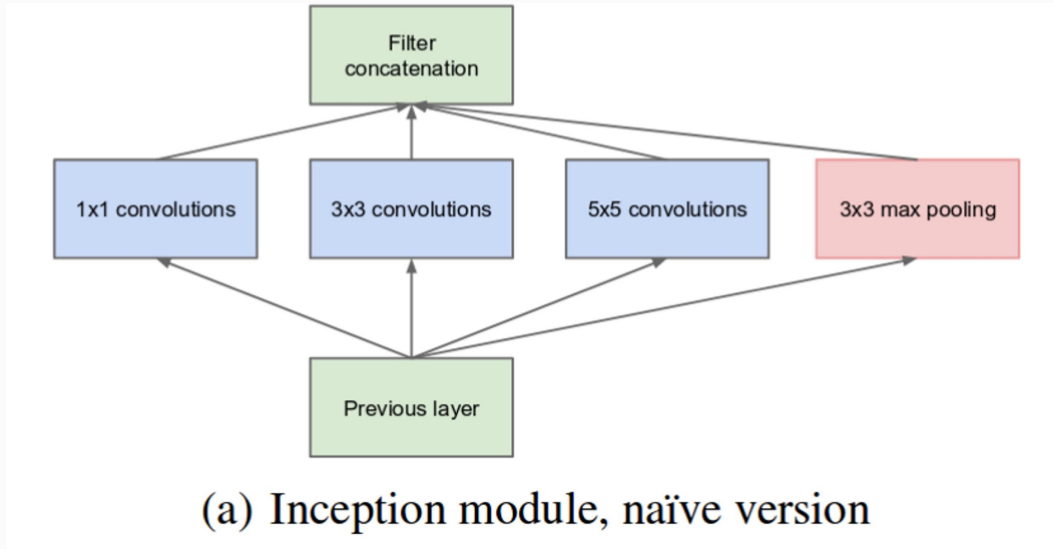
- During backpropagation:

$$\frac{dLoss}{dx} = \frac{dLoss}{d(F(x) + x)} = \frac{dLoss}{dF(x)} \cdot \frac{dF(x)}{dx} + 1$$

- That **+1** comes from the skip path.
- Even if $\frac{dF(x)}{dx}$ is tiny, the gradient **still has 1** → it doesn't vanish.

If transformation layers are not useful, network can simply **pass the input through unchanged**.

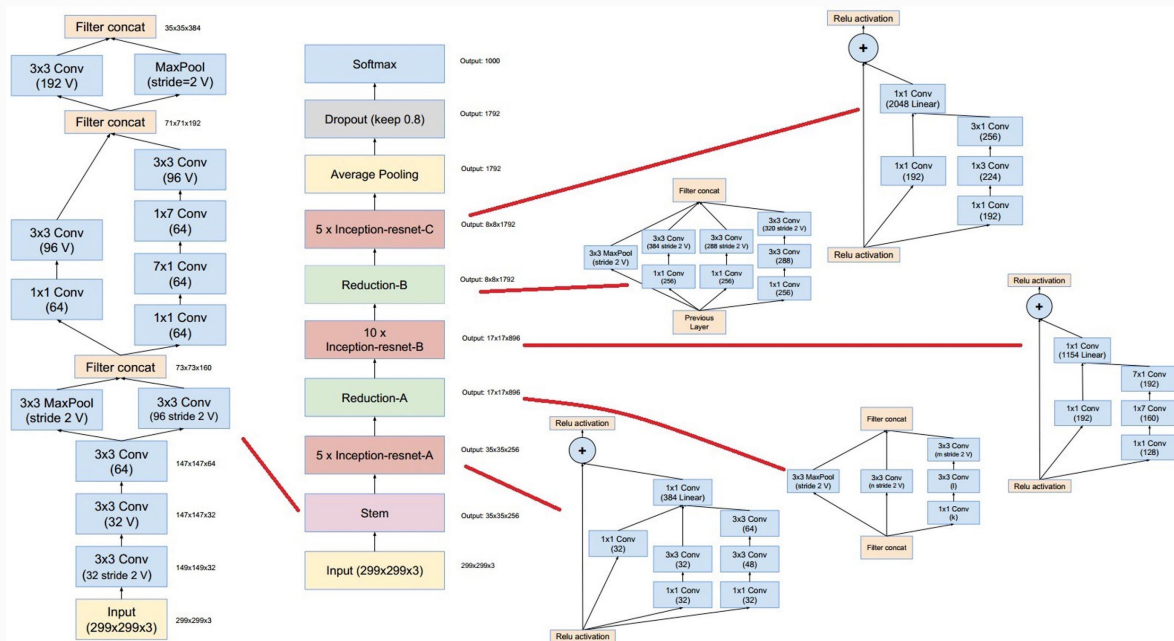
Inception and other variants



- Standard CNNs used one kernel size per layer, limiting scale flexibility.
- But objects in images appear at different scales.
 - Example: small eyes, large object
- A single kernel size might not capture all scales effectively.
- Inception Network solve this problem.
 - Instead of choosing one kernel size, they use multiple kernel sizes in the same layer.

Inception

- Inception networks make CNNs wider instead of just deeper.
- Multiple filters per layer → flexible and richer feature extraction.
- Because features are detected at multiple sizes, the network can recognize objects even if their size changes.



The trade-off of using Inception Network is mainly higher computational cost and architectural complexity compared with a standard CNN layer

Pre-Trained Models

- Inception Networks and similar architectures are **theoretically powerful**.
- **Real power** comes from using **pre-trained networks** for new tasks.
- Core idea: Neural networks, like software modules, can be **reused in appropriate scenarios**.
- **Transfer Learning** = transferring knowledge from a network trained on one task to a new task.
- The “pre-trained” network acts as a **starting point**, reducing the amount of data and training needed.
- Works for **images, text, and other domains**.

Transfer Learning

How Transfer Learning Works

- Step 1: Use a network pre-trained on a large dataset (e.g., ImageNet).
- Step 2: Remove the original head (the final classification layer).
- Step 3: Add a new head for your specific problem.
- Step 4: Train the new head, optionally fine-tune the rest of the network.

Benefits of Pre-Trained Networks

- Leverage knowledge from large datasets without training from scratch.
- Speeds up training and improves performance.
- Modular approach allows reuse across tasks and domains.

Pretrained CNN

↓

Remove original output layer

↓

Add new output layer for your task

↓

Train on your dataset

Lab Work

- Check the notebook and work on the suggested tasks

Assessment

- Assessment
 - Starts at ~10:10am, 45min, 1 attempt to submit
 - Open only lecture notes & lecture notebooks

APPENDIX

```

import tensorflow as tf
from tensorflow.keras import layers

# Input: 64x64 RGB image
inputs = tf.keras.Input(shape=(64, 64, 3))

# -----
# First Convolution Layer
# -----
x = layers.Conv2D(
    filters=32,           # Depth: 32 feature maps
    kernel_size=(3,3),    # Kernel: 3x3 window
    strides=1,            # Stride: move 1 pixel at a time
    padding='same',       # Padding 'same': output has same spatial size as input (64x64)
    activation='relu'     # Activation function
)(inputs)

# Optional: Pooling after conv
x = layers.MaxPooling2D(
    pool_size=(2,2),      # Pooling window 2x2
    strides=2,            # Stride 2 → halves spatial size (64x64 → 32x32)
    padding='same'        # Keep output size divisible by stride
)(x)

# -----
# Second Convolution Layer
# -----
x = layers.Conv2D(
    filters=64,           # Depth: 64 feature maps
    kernel_size=(5,5),    # Kernel: 5x5 window → captures larger patterns
    strides=1,            # Stride: move 1 pixel at a time
    padding='same',       # 'same' padding keeps spatial size (32x32) same
    activation='relu'
)(x)

# Optional: Pooling after second conv
x = layers.MaxPooling2D(
    pool_size=(2,2),      # Pooling window 2x2
    strides=2,            # Stride 2 → halves spatial size again (32x32 → 16x16)
    padding='same'
)(x)

```



Network Parameters

- **Kernel size:** defines the receptive field (3×3 = small, 5×5 = larger)
- **Filters / Depth:** number of feature maps
- **Stride:** how far the kernel moves (1 = fine scan)
- **Padding:** 'same' keeps the output spatial size equal to input; 'valid' would shrink it
- **Pooling:** reduces spatial dimensions, keeps depth same


```

# -----
# Flatten + Fully Connected Layers
# -----
x = layers.Flatten()(x)          # Convert 16x16x64 → 16384 neurons for dense layer
x = layers.Dense(128, activation='relu')(x) # Fully connected layer
outputs = layers.Dense(10, activation='softmax')(x) # Output layer for 10-class classification

# -----
# Build & Compile Model
# -----
from tensorflow.keras import models

model = models.Model(inputs=inputs, outputs=outputs)

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

# -----
# Model Summary
# -----
model.summary()

# -----
# Example Training (replace with your data)
# -----
model.fit(train_images, train_labels,
          validation_data=(val_images, val_labels),
          epochs=10, batch_size=32)

```

- **Flatten:** converts 3D feature maps → 1D vector for dense layers
- **Dense 128:** fully connected layer for learning higher-level features
- **Dense 10:** output layer for 10-class classification
- **Compile:** sets optimizer, loss, and metrics
- **Fit:** trains the network on your dataset

Practical Tips

Number of Layers

- Shallow CNNs (3–5 conv layers)
 - for small/simple datasets like MNIST or CIFAR-10.
- Moderate CNNs (5–10 conv layers)
 - common for medium datasets.
- Deep CNNs (50+ layers)
 - usually use residual blocks (ResNet) or Inception modules to avoid vanishing gradients

Rule of thumb: start small, then increase depth only if dataset and task require it.

Kernel (Filter) Size

- Common kernel sizes: 3×3 , 5×5 , 7×7
- Small kernels (3×3) are popular today:
 - Less parameters $\rightarrow$ easier to train
 - Can stack multiple layers of 3×3 to get effective larger receptive fields
- Large kernels (5×5 , 7×7) $\rightarrow$ capture bigger patterns in one layer, but more parameters.

Rule of thumb: start with 3×3 for most layers, use larger only at early layers if needed.

Number of Filters (Depth)

- Start small in early layers (e.g., 32–64)
 - capture simple patterns
- Increase as you go deeper (e.g., 128, 256, 512)
 - Pooling reduces spatial dimensions (width $\times$ height).
 - To compensate, we usually increase the number of filters.

Rule of thumb: double filters after each pooling layer